Лабораторная работа по криптографии №3
Жуховицкий А. Д. (М8О-303Б-23)
Этот Jupyter на GitHub

Метрика:

$$p = \frac{\text{число совпадений на одинаковых позициях}}{\text{длина текста}}$$

где `p` принимает значения от 0 до 1.

Алгоритм сравнения

1. **Нормализация:**
   - приводим тексты к одинаковому алфавиту (если это требуется);
   - приводим текст к нижнему регистру;
   - оставляем только буквенные символы (`isalpha()`), удаляя пробелы, знаки препинания и цифры.
2. **Посимвольное сравнение:**
   - проходим по позициям `i = 0..n-1`;
   - считаем, в скольких позициях `a[i] == b[i]`.
3. **Подсчёт доли совпадений:**
   - `p = matches / n`, где `n` — число букв.

Подготовка генераторов случайных данных

В качестве естественного языка я выбрал английский

Создадим "криптографически сильный псевдослучайный" генератор потока случайных букв

```
import secrets, string

def generate_random_letters(length: int) -> str:
    return "".join(secrets.choice(string.ascii_lowercase) for _ in range(length))
```

Создадим генератор слов используя библиотеку wonderwords, которая имеет немалый словарь с осмысленными словами.

```
from wonderwords import RandomWord, RandomSentence

r = RandomWord()
```

```python
def generate_mock_word(length: int) -> str:
    return r.word(word_min_length=length, word_max_length=length).lower()
```

Аналогично создадим генератор случайных осмысленных предложений

```python
s = RandomSentence()

def generate_mock_sentence() -> str:
    return "".join(char for char in s.sentence().lower() if char.isalpha())
```

Создадим генератор текста из случайных осмысленных слов

```python
def generate_random_word_text(length: int) -> str:
    sentence = []
    counter = 0
    while counter < length:
        if length - counter < 3:
            counter -= len(sentence.pop(-1))
            word = r.word(
                word_min_length=length - counter,
                word_max_length=length - counter
            )
            counter += len(word)
            sentence.append(word)
        else:
            word = r.word(word_max_length=min(15, length - counter))
            counter += len(word)
            sentence.append(word)
    return "".join(sentence).lower()
```

Создадим генератор осмысленного текста и генератор случайного текста

```python
def generate_mock_text(sentence_count: int) -> str:
    return "".join(generate_mock_sentence() for _ in range(sentence_count))


def generate_random_text(length: int) -> str:
    return generate_random_letters(length)
```

Создадим генератор RSA ciphertext в hex-алфавите

```python
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP

key         = RSA.generate(2048)
private_key = key
public_key  = key.publickey()
cipher      = PKCS1_OAEP.new(public_key)

def generate_random_rsa_cipher_text_hex(sentence_count: int) -> str:
    text = generate_random_text(sentence_count)
    ciphertext_bytes = cipher.encrypt(text.encode())
    return str(ciphertext_bytes.hex())
```

Создадим генератор AES ciphertext в hex-алфавите

```
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes

key    = get_random_bytes(32)   # AES-256
nonce  = get_random_bytes(8)
cipher = AES.new(key, AES.MODE_CTR, nonce=nonce)

def generate_random_aes_cipher_text_hex(sentence_count: int) -> str:
    text = generate_mock_text(sentence_count)
    return str(cipher.encrypt(text.encode()).hex())
```

Проверим

```
print(generate_random_letters(100))
print(generate_mock_word(8))
print(generate_mock_sentence())
print(generate_random_word_text(100))
print(generate_mock_text(3))
print(generate_random_text(100))
```

```
syelhddvxppxjvwpjmgdowkiifsfquyldvqwhapbqdjmxvhshpnqppladpfalgthqhyzxnmnuwidfrilukstk
huymcbzwjfpvsrw
timeline
theroughscaffoldseparatesbush
geneticstubachildmoleculenestconewomanlyexcellentprimaryparliamentcolonialfriendlyrip
eflintdetention
anonerouscongressdislikesperpanentertainingleopardringsmoaianiratesushiupdateshook
laftvflxldpfkpinqqbboxddqvvqrggbsqrpvmmisusfptglzskddfhbypjwsidnmmsdodmavnsgsdcvnvxjo
obeuhohssneqord
```

Сравнение

Сравним 2 осмысленных текста на естественном языке

```
def compare_text_to_text(sentence_count: int) -> float:
    text1 = generate_mock_text(sentence_count)
    text2 = generate_mock_text(sentence_count)

    # Выравнивание длины текста
    min_len = min(len(text1), len(text2))
    text1 = text1[:min_len]
    text2 = text2[:min_len]

    return sum(text1[i] == text2[i] for i in range(len(text1))) / len(text1)
```

```
from utils.lab3 import compare_text_to_text

print("Пример текстов:")
compare_text_to_text(5, True)
```

```
Пример текстов:
text1: adamppilotalteredsporchthecombativesubjectthawslawyertheearsplittingpicturefen
cesstereogustymeteorologyinformsdramaanappycovera
text2: ahaplessroastsmellsjacketanaberrantlesbiandivertsfactoryfranticyarmulkecomplet
esvineclumsymisterseeksremainsgoodthighfencesplay
```

```
0.06299212598425197
```

```python
from utils.lab3 import run_single_text_comparison
from concurrent.futures import ProcessPoolExecutor, as_completed
from collections import defaultdict


times: int = 1000
workers: int = 16


results = defaultdict(float)
counts  = defaultdict(int)

sentence_counts = range(5, 20, 2)
tasks = [sc for sc in sentence_counts for _ in range(times)]

with ProcessPoolExecutor(max_workers=workers) as executor:
    futures = {executor.submit(run_single_text_comparison, sc): sc for sc in tasks}

    for future in as_completed(futures):
        sc, match = future.result()
        results[sc] += match
        counts[sc]  += 1

for sc in sorted(results):
    avg = results[sc] / counts[sc]
    print(f"Для {sc} осмысленных предложений: {avg:.2%}"
          f" совпадения в среднем за {times} генераций")
```

```
Для 5 осмысленных предложений: 7.10% совпадения в среднем за 1000 генераций
Для 7 осмысленных предложений: 6.99% совпадения в среднем за 1000 генераций
Для 9 осмысленных предложений: 6.84% совпадения в среднем за 1000 генераций
Для 11 осмысленных предложений: 6.81% совпадения в среднем за 1000 генераций
Для 13 осмысленных предложений: 6.68% совпадения в среднем за 1000 генераций
Для 15 осмысленных предложений: 6.70% совпадения в среднем за 1000 генераций
Для 17 осмысленных предложений: 6.68% совпадения в среднем за 1000 генераций
Для 19 осмысленных предложений: 6.61% совпадения в среднем за 1000 генераций
```

Сравним осмысленный текст и текст из случайных букв

```python
def compare_text_to_random_letters(sentence_count: int) -> float:
    text1 = generate_mock_text(sentence_count)
    text2 = generate_random_letters(len(text1))
    return sum(text1[i] == text2[i] for i in range(len(text1))) / len(text1)
```

```python
from utils.lab3 import compare_text_to_random_letters

print("Пример текстов:")
compare_text_to_random_letters(5, True)
```

```
Пример текстов:
text1: atartstrategysolvesbeginningtheselfishrecognitionbangsshoehornthegorgeousgastr
opodmatchesthousandacrowdedbrandyinstallscropatinytenemententerscushion
text2: cepfxlfyhnuauqepwotttwsddvvzxnyekzwdcoscrrgwspiyxsyelihhcktecwrxnaqughoywjzrkw
wvuzuvzzvctxdqajfsjgvtwvjyoaxmhoskstpnmaggsxacqmvleyydzkpfhtlgpbmdtyqkx
```

```
0.03355704697986577
```

```python
from utils.lab3 import run_single_text_to_random_letters_comparison


times = 1000
workers = 16
results = defaultdict(float)
counts  = defaultdict(int)

tasks = [sc for sc in range(5, 20, 2) for _ in range(times)]

with ProcessPoolExecutor(max_workers=workers) as executor:
    futures = {
        executor.submit(run_single_text_to_random_letters_comparison, sc):
            sc for sc in tasks
    }
    for future in as_completed(futures):
        sc, match = future.result()
        results[sc] += match
        counts[sc]  += 1

for sc in sorted(results):
    avg = results[sc] / counts[sc]
    print(f"Для {sc} осмысленных предложений: {avg:.2%}"
          f" совпадения в среднем за {times} генераций")
```

```
Для 5 осмысленных предложений: 3.79% совпадения в среднем за 1000 генераций
Для 7 осмысленных предложений: 3.84% совпадения в среднем за 1000 генераций
Для 9 осмысленных предложений: 3.89% совпадения в среднем за 1000 генераций
Для 11 осмысленных предложений: 3.83% совпадения в среднем за 1000 генераций
Для 13 осмысленных предложений: 3.79% совпадения в среднем за 1000 генераций
Для 15 осмысленных предложений: 3.88% совпадения в среднем за 1000 генераций
Для 17 осмысленных предложений: 3.89% совпадения в среднем за 1000 генераций
Для 19 осмысленных предложений: 3.87% совпадения в среднем за 1000 генераций
```

Сравним осмысленный текст и текст из случайных слов

```python
def compare_text_to_random_word_text(sentence_count: int) -> float:
    text1 = generate_mock_text(sentence_count)
    text2 = generate_random_word_text(len(text1))
    return sum(text1[i] == text2[i] for i in range(len(text1))) / len(text1)
```

```python
from utils.lab3 import compare_text_to_random_word_text

print("Пример текстов:")
compare_text_to_random_word_text(5, True)
```

```
Пример текстов:
text1: theboilingmortiseblushesblushseemlyautoreflectscucumberthestrongbonusannoysswi
mawhimsicalfiberburnspantherwretchedparserpleasesexclamation
text2: muscatelmarchcharsetcolonthiefsonnetdraftstrategyfundraisingpossibilitymemoryq
uacktrayremnantphosphatepeacefulceleriackeyboardeminentyouth
```

```
0.043478260869565216
```

```python
from utils.lab3 import run_single_text_to_random_word_text_comparison


times = 1000
workers = 16
results = defaultdict(float)
counts  = defaultdict(int)
tasks = [sc for sc in range(5, 20, 2) for _ in range(times)]

with ProcessPoolExecutor(max_workers=workers) as executor:
    futures = {
        executor.submit(run_single_text_to_random_word_text_comparison, sc):
            sc for sc in tasks
    }
    for future in as_completed(futures):
        sc, match = future.result()
        results[sc] += match
        counts[sc]  += 1

for sc in sorted(results):
    avg = results[sc] / counts[sc]
    print(f"Для {sc} осмысленных предложений: {avg:.2%} "
          f"совпадения в среднем за {times} генераций")
```

```
Для 5 осмысленных предложений: 6.33% совпадения в среднем за 1000 генераций
Для 7 осмысленных предложений: 6.30% совпадения в среднем за 1000 генераций
Для 9 осмысленных предложений: 6.27% совпадения в среднем за 1000 генераций
Для 11 осмысленных предложений: 6.22% совпадения в среднем за 1000 генераций
Для 13 осмысленных предложений: 6.24% совпадения в среднем за 1000 генераций
Для 15 осмысленных предложений: 6.30% совпадения в среднем за 1000 генераций
Для 17 осмысленных предложений: 6.23% совпадения в среднем за 1000 генераций
Для 19 осмысленных предложений: 6.31% совпадения в среднем за 1000 генераций
```

Сравним 2 текста из случайных букв

```python
def compare_random_text_to_random_text(letter_count: int) -> float:
    text1 = generate_random_letters(letter_count)
    text2 = generate_random_letters(letter_count)
    return sum(text1[i] == text2[i] for i in range(letter_count)) /
letter_count
```

```python
from utils.lab3 import compare_random_text_to_random_text

print("Пример текстов:")
compare_random_text_to_random_text(100, True)
```

```
Пример текстов:
text1: sjqcqnupmidnjhyvgnfpnhyfeafkkgnibnpqhkpoydgmvqhmquvdijvwmnnttipeanncwxxtzcnjvy
nekahxxruvtiayqluzhmtz
text2: nfexrnpaqqrrvprnsfcklhyqrxutezpdbcsavwsevyennfqwxamroepfhrnqiezetbhripmdtmcpvr
nquajnrutezgnbqbqwrmbq
0.11
```

```python
from utils.lab3 import run_single_random_text_comparison


times = 1000
workers = 16
results = defaultdict(float)
counts  = defaultdict(int)

tasks = [lc for lc in (10, 100, 1000, 2000) for _ in range(times)]

with ProcessPoolExecutor(max_workers=workers) as executor:
    futures = {
        executor.submit(run_single_random_text_comparison, lc):
            lc for lc in tasks
    }
    for future in as_completed(futures):
        lc, match = future.result()
        results[lc] += match
        counts[lc]  += 1

for lc in sorted(results):
    avg = results[lc] / counts[lc]
    print(f"Для {lc} букв: {avg:.2%} совпадения в среднем за {times} генераций")
```

```
Для 10 букв: 3.59% совпадения в среднем за 1000 генераций
Для 100 букв: 3.79% совпадения в среднем за 1000 генераций
Для 1000 букв: 3.84% совпадения в среднем за 1000 генераций
Для 2000 букв: 3.84% совпадения в среднем за 1000 генераций
```

Сравним 2 текста из случайных слов

```python
def compare_random_word_text_to_random_word_text(letter_count: int) ->
float:
    text1 = generate_random_word_text(letter_count)
    text2 = generate_random_word_text(letter_count)
    return sum(text1[i] == text2[i] for i in range(letter_count)) /
letter_count
```

```python
from utils.lab3 import compare_random_word_text_to_random_word_text

print("Пример текстов:")
compare_random_word_text_to_random_word_text(100, True)
```

```
Пример текстов:
text1: nestwantfordsuittakeoverbowerfedorafamilysensitiveviolagazemarriagesecuritybug
lecampaigncivilization
text2: functionalitytickarrogantownershipcollagenlaggathersubprimevelocityshakertrowe
lthreatingredientsnarl
```

0.12

```
from utils.lab3 import run_single_random_word_text_comparison


times = 1000
workers = 16
results = defaultdict(float)
counts  = defaultdict(int)

tasks = [lc for lc in (20, 100, 1000, 2000) for _ in range(times)]

with ProcessPoolExecutor(max_workers=workers) as executor:
    futures = {
        executor.submit(run_single_random_word_text_comparison, lc):
            lc for lc in tasks
    }
    for future in as_completed(futures):
        lc, match = future.result()
        results[lc] += match
        counts[lc]  += 1

for lc in sorted(results):
    avg = results[lc] / counts[lc]
    print(f"Для {lc} букв: {avg:.2%} совпадения в среднем за {times} генераций")
```

```
Для 20 букв: 7.00% совпадения в среднем за 1000 генераций
Для 100 букв: 6.38% совпадения в среднем за 1000 генераций
Для 1000 букв: 6.24% совпадения в среднем за 1000 генераций
Для 2000 букв: 6.25% совпадения в среднем за 1000 генераций
```

Проверка гипотезы

Также сравним насколько RSA и AES ciphertext в hex похожи на случайный hex и что больше похоже на случайный hex: он сам или RSA и AES

```
def compare_rsa_hex_ciphertext_to_random_hex(
        sentence_count: int
) -> tuple[int, float]:
    ciphertext_hex = generate_random_rsa_cipher_text_hex(sentence_count)
    random_text_hex = secrets.token_hex(len(ciphertext_hex) // 2)

    return (
        len(ciphertext_hex),
        sum(ciphertext_hex[i] == random_text_hex[i]
            for i in range(len(ciphertext_hex))) / len(ciphertext_hex)
    )
```

```
def compare_aes_hex_ciphertext_to_random_hex(
    sentence_count: int
) -> tuple[int, float]:
    ciphertext_hex = generate_random_aes_cipher_text_hex(sentence_count)
    random_text_hex = secrets.token_hex(len(ciphertext_hex) // 2)
```

```python
    return (
        len(ciphertext_hex),
        sum(ciphertext_hex[i] == random_text_hex[i]
            for i in range(len(ciphertext_hex))) / len(ciphertext_hex)
    )
```

```python
def compare_random_hex_to_random_hex(length: int) -> float:
    a = secrets.token_hex(length // 2)
    b = secrets.token_hex(length // 2)

    return sum(a[idx] == b[idx] for idx in range(length)) / length
```

```python
times = 1000

for sentence_count in range(5, 10):
    avg_cipher = 0
    avg_random = 0
    for _ in range(times):
        length, percent = compare_rsa_hex_ciphertext_to_random_hex(sentence_count)
        avg_cipher += percent
        percent = compare_random_hex_to_random_hex(length)
        avg_random += percent

    print(f"Для {sentence_count} предложений: RSA ciphertext "
          f"{avg_cipher / times:.2%} и Random hex {avg_random / times:.2%}"
          f" совпадения с Random Hex в среднем за {times} генераций")
```

```
Для 5 предложений: RSA ciphertext 6.18% и Random hex 6.37% совпадения с Random Hex в
среднем за 1000 генераций
Для 6 предложений: RSA ciphertext 6.62% и Random hex 6.13% совпадения с Random Hex в
среднем за 1000 генераций
Для 7 предложений: RSA ciphertext 6.17% и Random hex 6.10% совпадения с Random Hex в
среднем за 1000 генераций
Для 8 предложений: RSA ciphertext 6.43% и Random hex 6.28% совпадения с Random Hex в
среднем за 1000 генераций
Для 9 предложений: RSA ciphertext 6.27% и Random hex 6.23% совпадения с Random Hex в
среднем за 1000 генераций
```

```python
times = 1000

for sentence_count in range(5, 10):
    avg_cipher = 0
    avg_random = 0
    for _ in range(times):
        length, percent = compare_aes_hex_ciphertext_to_random_hex(sentence_count)
        avg_cipher += percent
        percent = compare_random_hex_to_random_hex(length)
        avg_random += percent

    print(f"Для {sentence_count} предложений: AES ciphertext "
          f"{avg_cipher / times:.2%} и Random hex {avg_random / times:.2%}"
          f" совпадения с Random Hex в среднем за {times} генераций")
```

```
Для 5 предложений: AES ciphertext 6.24% и Random hex 6.32% совпадения с Random Hex в
среднем за 1000 генераций
Для 6 предложений: AES ciphertext 6.25% и Random hex 6.22% совпадения с Random Hex в
среднем за 1000 генераций
Для 7 предложений: AES ciphertext 6.30% и Random hex 6.23% совпадения с Random Hex в
среднем за 1000 генераций
Для 8 предложений: AES ciphertext 6.26% и Random hex 6.22% совпадения с Random Hex в
среднем за 1000 генераций
Для 9 предложений: AES ciphertext 6.26% и Random hex 6.18% совпадения с Random Hex в
среднем за 1000 генераций
```

Можно заметить, что RSA и AES шифры довольно незаметны среди просто случайного hex потока, что показывает скрытность факта шифрования в случае hex'a

Итого:

- два осмысленных текста на естественном языке - ~6,85% совпадения
- осмысленный текст и текст из случайных букв - ~3,85% совпадения
- осмысленный текст и текст из случайных слов - ~6,30% совпадения
- два текста из случайных букв - ~3,83% совпадения
- два текста из случайных слов - ~6,30% совпадения